Tech Stack Overview

This document summarizes the technologies currently used in the repository and where they are defined.

Monorepo layout

- **apps/web/**: Next.js (frontend)
- **apps/api/**: FastAPI (backend API)
- **apps/worker/**: Python worker (RQ jobs)
- **apps/pdf/**: Express + pdfkit microservice
- **packages/ui/**: Shared React UI package
- **docker-compose.yml**: Local dev stack
- Frontend auth: apps/web/app/api/auth/[...nextauth]/route.ts
- API entry: apps/api/app/main.py
- DB models: apps/api/app/models.py
- Suggestions: apps/api/app/suggestions_engine.py
- Web middleware: apps/web/middleware.ts
- Prisma schema: apps/web/prisma/schema.prisma

Frontend (Web)

- Framework: Next.js 14 (App Router) + React 18
- Language: TypeScript 5
- Auth: NextAuth v4 (Email + Credentials), Prisma adapter
- ORM (auth store): Prisma Client 5
- Styling: TailwindCSS 3, PostCSS, autoprefixer
- Data fetching: @tanstack/react-query
- Notifications: react-hot-toast
- Charts: Recharts
- Shared UI: @dea/ui
- Auth gating: next-auth middleware redirects to /signin
- Bearer propagation: Authorization: Bearer <token> from session
- Docker: node:20-alpine + openssl1.1-compat; prisma generate before build

Backend API

- Framework: FastAPI; ASGI: Uvicorn
- ORM: SQLAlchemy 2; Migrations: Alembic
- Database: PostgreSQL (postgres:16-alpine)
- Auth/JWT: PyJWT; Password hashing: passlib[bcrypt]
- Validation: Pydantic v2 (+ pydantic-settings)
- Queue: redis + rq
- HTTP client: httpx (PDF service calls)
- Uploads/JSON: python-multipart, orjson
- Security: owner scoping (BOLA), rate limit (Redis), security headers
-

Key endpoints: auth, transactions, spend summary, suggestions, debt simulate/export, flags, health

- Docker: python:3.11-slim

Background worker

- Python with redis, rq, SQLAlchemy, psycopg
- Docker image configured in apps/worker/Dockerfile

PDF microservice

- Node + Express with pdfkit
- Endpoint consumed by API (PDF_SERVICE_URL)
- Port: container 4000 (host http://localhost:4001/)

Infrastructure / DevOps

- Docker Compose services: db, redis, minio, mailhog, pdf, api, worker, web
- Port mappings (host !’ container): Postgres 5433!’5432, Redis 6380!’6379, MailHog 8026!’8025, MinIO 9002!’9000 (console 9003!’9001), PDF 4001!’4000, API 8000!’8000, Web 3000!’3000
- Storage: MinIO S3-compatible store
- Email: MailHog for dev email capture

Testing

- pytest (API tests under apps/api/tests/)

Languages and runtimes

- TypeScript/JavaScript (Node 20) for Web and PDF
- Python 3.11 for API and Worker
- PostgreSQL via SQLAlchemy and Prisma

Quick Links

- Home: http://localhost:3000/
- Sign in: http://localhost:3000/signin
- Sign up: http://localhost:3000/signup
- Spend: http://localhost:3000/spend
- Suggestions: http://localhost:3000/suggestions
- Debt: http://localhost:3000/debt
- Flags: http://localhost:3000/flags
- API Health: http://localhost:8000/healthz
- MailHog: http://localhost:8026/
- PDF service: http://localhost:4001/
- MinIO: http://localhost:9002/ (console http://localhost:9003/)

Generated on 21/10/2025, 6:59:31 pm